

Device / System-Side Update Client — Production + Multi-Account Design (to-be)

Revision: 1 Last modified: 2026-07-10T11:18:54Z

Design proposal (§11.4.6 honest boundary applies — see the closing section). This is a **to-be** design for the device/System-side update client, extending it for multi-account. It does **not** write code and does **not** contradict the authoritative as-is inventory 11_existing_upload_and_device_update.md (hereafter **11_***) — every current-state claim below is cited to 11_* (which itself cites file:line). The mandate framing is 00_INDEX.md item 10 (“device/System-side update client — contacts the configured Helix OTA system, receives new-OTA-update notifications, informs users of a new System version via a setup wizard; *we have it already most likely* → catalogue-first discovery; extend to production + full test coverage”). Account / project / device-identity **shapes** are owned by 20_target_multitenancy_data_model.md as the single source of truth (SSOT); at the time of writing that file is **planned but not yet authored** (00_INDEX §3), so this doc uses only the direction 00_INDEX already fixes — account → projects → OTA updates, a **server-minted token account/project claim**, and account_id/project_id scoping columns — and defers every canonical entity field to 20_*. It invents no conflicting shape.

Anchoring facts from 11_* (current state — do not contradict): the device client **EXISTS** = submodules/ota-android-agent, a **headless WorkManager poll** worker (15-min periodic + jitter/backoff, PollScheduler.kt, README.md:46) that hits GET /client/update and runs one cycle poll → download → **verify-before-apply** → **auto-apply** → telemetry (OtaPollWorker.kt:59-126, auto-apply :116-124). The device identifies itself **only** by its bearer-token subject deviceId (handlers_client.go:22-23; agent Dtos.kt:33-36) — **no account/project is ever transmitted**; the device token is operator-minted (mintDeviceToken role=device sub=deviceId, handlers_device.go:157-159; registration requires an operator/admin

token, server.go:192). Update-check resolution keys on (os, model, group) only (ActiveDeploymentForTarget, handlers_client.go:43). Token claims are {sub, roles, iat, exp} — **no account/project claim** (token.go:31-36). There is **no push/notification** (poll-only, 11_* §5) and **no device-side setup/consent “new version” wizard** (11_* §6). The concrete HTTP client + device config are **not in the submodule** — ControlPlaneClient/Downloader/Verifier/Telemetry are **interfaces only** (Ports.kt); the only concrete reference client is the Go emulator server/cmd/ota-device-emu (11_* §4, honest gap 2).

1. Device identity + scoping to (account, project)

1.1 Today — a bare deviceId subject, globally scoped

Per 11_* the device carries a single opaque bearer token whose sub is the deviceId; nothing else identifies it. The server never filters by account or project, so **every device is scoped only by (os, model, group) against the whole control plane**. In a multi-account deployment this is a cross-tenant leak by construction: a device enrolled by account A, if its (os, model, group) matches, would be offered account B’s active deployment — because no data-level account boundary exists (11_* §7, §Honest-gaps 3-4).

1.2 Target — a server-minted (account, project, device) identity

The design goal fixed by 00_INDEX is account → projects → OTA updates with the token carrying an account/project claim. Applied to the device client:

- The device token’s claims gain **account_id + project_id** (extending Claims{sub,roles,iat,exp}, token.go:31-36) — **exact field names/types are owned by 20_***; this doc only asserts the *presence* and *direction* of the claim.
- store.Device gains the scoping columns 20_* defines (today it has none, store.go:35-56); mintDeviceToken stamps the claim at registration.
- GET /client/update reads the claim and calls an **account/project-scoped** deployment lookup (extend ActiveDeploymentForTarget

with the account+project filter). A device then **only ever sees its own account's updates**.

- **Trust boundary (mirrors `resolvePublicKey, 11_* §3`):** the account/project claim is **server-minted and server-verified** — the device can never assert or override its own tenancy in a request. This is the same trust posture the artifact-verify key already uses (key from server config only), applied to tenancy.

1.3 Provisioning the credential — two options, with a recommendation

How does a device *get* an (account, project)-scoped credential at first boot?

Option	Mechanism	Pros	Cons
A — Operator-minted scoped token (extend today's flow)	Enrollment stays operator-driven (as today, <code>handleRegisterDevice</code> requires operator/admin, <code>server.go:192</code>); <code>mintDeviceToken</code> simply also stamps <code>account_id/project_id</code> .	Minimal delta over the <i>existing</i> flow; no new enrollment protocol; matches today's "operator registers the device" reality.	Operator must touch every device (or script it); no self-service; a long-lived token on-device is higher blast-radius if leaked.
B — Bootstrap-claim → per-device credential (fleet-provisioning pattern)	Device ships with a short-lived, account+project-scoped enrollment/claim token (limited to "register me"); at first boot it exchanges the claim for a long-lived per-device token carrying the full (account, project, deviceId) claim. Server runs a provisioning-hook validation before	Scales to many devices / self-service; the on-device long-lived secret is minted per-device and rotatable; the shared bootstrap secret has near-zero privilege.	New enrollment endpoint + claim-token lifecycle; more moving parts; hook validation to build.

Option	Mechanism	Pros	Cons
	issuing. Models AWS IoT <i>fleet provisioning by claim</i> .		

Recommendation: start with Option A, because enrollment is *already* operator-driven today (11_* §4) — the production delta is just “stamp the claim,” which is the smallest safe change and reuses the existing register path. **Adopt Option B’s bootstrap → rotate model when at-scale or self-service enrollment is required** (the two are not mutually exclusive — B is A plus a claim-exchange front door). This is a recommendation, not a silent decision: 20_*/21_* own the final token/enrollment shape and may choose B first if self-service is a launch requirement. **Open question for the operator:** is device enrollment operator-only at launch (favours A) or self-service (favours B)?

2. Update-check transport — keep poll, add a push “wake”

2.1 Today — poll only

Per 11_* §4-§5: a 15-min WorkManager periodic worker with jitter + exponential backoff polls GET /client/update → 200 UpdateAvailable | 204 no-update | transient error. There is **no** push/FCM/WebSocket/SSE channel anywhere in server/internal or the agent (11_* §5). Latency to a new release is therefore bounded by the poll interval (~15 min worst case).

2.2 The tradeoff (external research, §11.4.8)

Axis	Poll (pull)	Push (FCM / SSE / WebSocket)
New-release latency	Bounded by interval (~15 min)	Seconds
Battery	Periodic radio wake per device	One shared idle socket (FCM) is near-zero cost; a bespoke WebSocket is a persistent connection

Axis	Poll (pull)	Push (FCM / SSE / WebSocket)
Server load at fleet scale	Thundering-herd risk unless jittered (“floating cycle” so N devices don’t poll at once)	Fan-out cost moves to the push broker; server just enqueues
Offline tolerance	Naturally retries next cycle; robust	Push is best-effort; a missed push must still be caught by a poll
Reachability	Works anywhere HTTP works	FCM needs Google Play Services — RK3588 / Orange Pi 5 Max AOSP targets may be non-GMS → FCM not guaranteed
Complexity	Already built	New channel + per-device addressing keyed on (account, project, deviceId)

Sources: Mender OTA best-practices + saince.io (pull = less bandwidth but interval-bound latency; unsafe-state deferral), FSS/Hubble (fleet-scale thundering-herd → floating/jittered cycle; FCM single shared socket is battery-efficient instant delivery).

2.3 Recommendation — hybrid: poll is the floor, push is a latency hint

The mandate (“receive new-OTA-update notifications from the System”) argues for push, but push must **never** become the source of truth. Recommended design:

1. **Keep the 15-min jittered poll as the reliable floor** — it already exists, it is offline-tolerant, and it is the *only* path that works on non-GMS AOSP targets. poll + VerifyBeforeApply stays the **single source of truth** for “is there an update, and is it valid.”
2. **Add an optional push “wake” channel** that carries **no update payload** — it only tells the device “*something changed, poll now,*” collapsing latency from ~15 min to seconds. The device responds to a wake by running its normal authenticated poll + verify. A dropped/spoofed wake is harmless: worst case the device just polls

on its next scheduled cycle; a spoofed wake cannot inject an update because the poll+verify path is unchanged.

3. **Transport per device class:** FCM where Play Services is present; **SSE or a long-poll/WebSocket wake fallback for non-GMS RK3588/Orange Pi targets** (the project's primary hardware). Server seam: a new notify fan-out keyed on (account, project, deviceId) triggered when a deployment is created/updated for that scope (handleCreateDeployment is the emit point).

This is a recommendation-with-tradeoffs: an operator who values simplicity over latency can ship **poll-only** first (it already works) and add the wake channel as a fast-follow — the wake is purely additive and the design must not regress to “push-only.”

3. New-version notification → the user

Per 11_* §6 this is **absent** today: the agent is headless and **auto-applies** without any user-facing “a new version is available” surface (OtaPollWorker.kt:116-124).

Design. When a poll (or a push wake → poll) returns 200 UpdateAvailable **and the account/project policy is interactive** (see §3.1), the client:

1. Posts an **Android system notification** (dedicated notification channel, default importance) — title “System update available”, body with target **version**, **download size**, and a short **release-notes** line.
 - *Server dependency (flag):* the release record carries no notes field today (11_* §1); surfacing human-readable notes needs a small server addition (a notes field on the release + pass-through in the UpdateAvailable offer, handlers_client.go:70-101). Until then the notification shows version + size only. Stated as a fact, not a guess (§11.4.6).
2. Respects **deferral conditions** — quiet hours, metered-network, low battery, low storage — so the notification/apply does not fire at a hostile moment.
3. On tap → opens the **setup wizard** (§4).

3.1 Two policy modes (per account/project)

Because some tenants run **unattended kiosks** (no human at the device) and others run **user-attended** devices, notification/consent is a **per-account/project policy**, provisioned at enrollment:

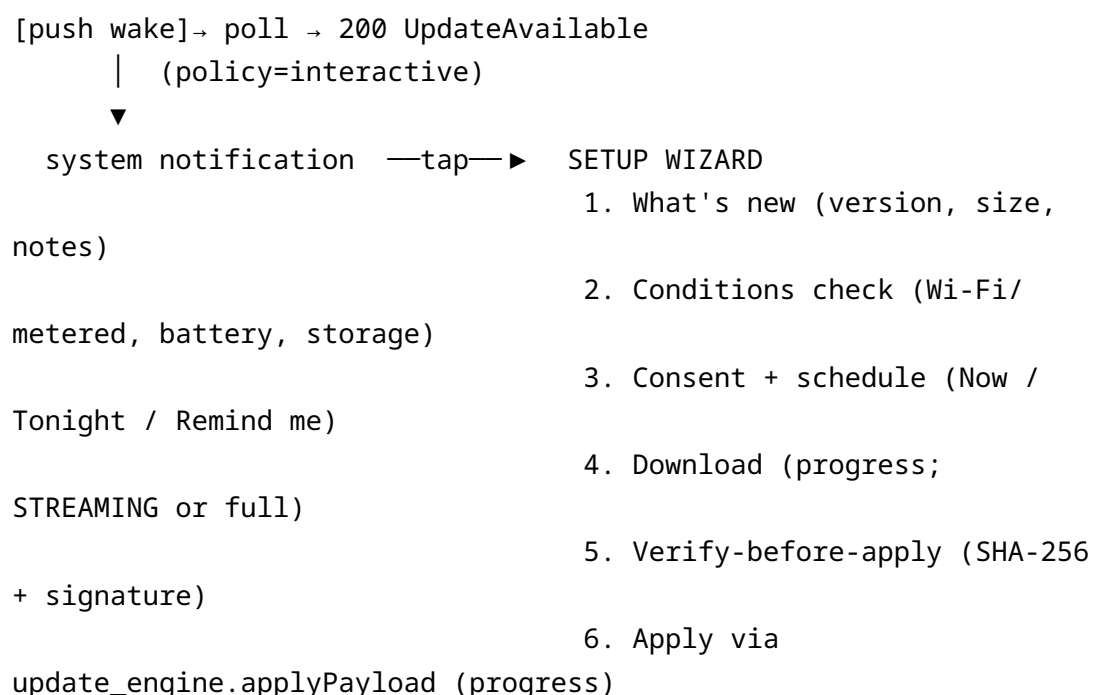
- **silent / auto** — today's headless behaviour: verify + auto-apply, no notification, no wizard. Correct for kiosk/signage/unattended fleets. The "consent" is the account policy the operator set.
- **interactive / consent** — notify the user, run the wizard, apply only on consent. Correct for user-facing devices.

This keeps the existing headless path valid (no regression) while adding the user-informing surface the mandate asks for.

UI note: the notification's expanded content and any activity it opens are a **rendered UI surface** → they MUST use the **OpenDesign design tokens (§11.4.162)** (light + dark) and are covered by the §11.4.170 host-rendered visual proof described in §4 — the Android agent is **headless today**, so this is net-new UI.

4. Setup wizard — notify → wizard → consent → apply → A/B update_engine handoff

4.1 Flow



7. Reboot to switch slot (A/B seamless; auto-rollback on bad boot)

Steps 4-7 reuse the agent's **existing** one-cycle path (`OtaPollWorker.runCycle`, `OtaPollWorker.kt:59-126`) — the wizard simply **gates** that path on user consent instead of auto-running it.

4.2 `update_engine` handoff (grounded in the AOSP `SystemUpdaterSample` pattern)

The Android A/B reference client (`SystemUpdaterSample`) is a **pull** app that lists available updates and, on the user's **Apply**, calls `UpdateEngine#applyPayload(payloadURL, offset, size, headers)` — passing the Authorization/User-Agent headers — and receives `onStatusUpdate` / `onPayloadApplicationComplete` callbacks; it persists its own updater state so it survives disconnects/resume. Two transfer modes: **non-streaming** (whole package URL handed to the engine) and **STREAMING** (only ZIP entries fetched first; `payload.bin` streamed by `update_engine` directly, entries stored uncompressed for offset access). Helix's wizard maps onto exactly this:

- **Download/apply:** hand the artifact URL (`ArtifactBaseURL/<id>.zip`, `handlers_client.go:232-234`) + the device's bearer Authorization header to `applyPayload`; prefer **STREAMING** to avoid double-storing the payload.
- **Verify-before-apply gate stays mandatory:** the agent's `VerifyBeforeApply` re-checks SHA-256 + signature ordered `MALFORMED_DIGEST` → `HASH_MISMATCH` → `SIGNATURE_INVALID` (11_* §3), and a rejected artifact never reaches `update_engine` (`OtaPollWorker.kt:99-113`). The signature verified is the one the server persisted and handed to the device (`handlers_artifact.go:196` → `handlers_client.go:81`).
- **A/B seamless + rollback:** apply happens to the inactive slot in the background; the wizard's final step is a **reboot-to-switch** prompt. Per the project overview (`root CLAUDE.md`) the target uses `update_engine` + AVB/dm-verity
 - **auto-rollback** — a failed boot rolls back automatically, so the wizard must present reboot honestly (“your device will restart to finish; it will roll back automatically if anything is wrong”) and never brick (§11.4.133 **target/hardware safety**).

4.3 First-run enrollment sub-wizard

On the **very first boot** the wizard also runs enrollment (§1): receive/enter the account+project claim (Option A) or exchange the bootstrap claim (Option B), register the device, and store the minted per-device token securely (§5). After enrollment the device is scoped and the normal update flow applies.

4.4 This is the device client's FIRST UI surface — mandates

The agent + ota-update-engine-bridge are **headless today** (11_* §4, §6). The wizard is therefore a **net-new UI module** — a companion activity / Compose UI added to the :android layer (or a small companion app). As a rendered UI surface it **MUST**:

- Use the **OpenDesign design tokens (§11.4.162)** — light and dark variants, no ad-hoc styling, no label overlap.
- Be proven by **device-independent host-rendered pixel proof (§11.4.170)** — the real Compose screens rendered to PNG on the host (Paparazzi/Roborazzi class) for **every screen × state × {light, dark}**, dual-validated by golden image-diff and an OCR/vision layout oracle. Value/token-equality unit tests are **forbidden as the sole UI proof**.

Kiosk fallback: where policy = silent (no user), the wizard is bypassed and the headless auto-apply path runs unchanged (§3.1) — an honest, operator-configured skip, not a faked consent.

5. Production hardening — the concrete client + config that 11_* says are missing

11_* (§4, honest gap 2) establishes that the agent ships **interfaces only** (Ports.kt) — no concrete ControlPlaneClient/Downloader/Verifier/Telemetry, no server-URL/credential config; the only concrete reference is the Go ota-device-emu. A production impl needs:

1. **Concrete ControlPlaneClient** (over the project's HTTP transport): configurable **base URL**; **token storage** in Android Keystore / EncryptedSharedPreferences (never plaintext, §11.4.10); **token refresh** via /auth/refresh; the **account/project claim carried in**

- every request** via the token (never as a spoofable header — §1.2 trust boundary).
2. **Device config** — server URL, account/project enrollment material, poll interval, policy (silent/interactive), push-channel opt-in — sourced by **configuration injection** (env/config/enrollment payload), **never hardcoded** into the submodule (§11.4.28 decoupling — the agent stays project-agnostic).
 3. **Retry / backoff** — extend the existing jitter + exponential backoff (PollScheduler.kt) to the **download** and **telemetry** legs; poll is idempotent; a transient error retries, a 204 is success-no-op.
 4. **Offline handling (§11.4.144 availability-following)** — the device may drop off any transport at any time. The client **MUST**: detect the offline state, log an **honest offline event** (never present a stale “up to date” as live), **follow reconnection using the already-defined backoff** (never invent timings), **resume** an interrupted download (A/B streaming + persisted updater state — the SystemUpdaterSample resume pattern), and **re-attach + log resume** on return. Server side: deployment progress is **derived from telemetry** (deriveProgress, handlers_deployment.go:202), so an offline device simply stops reporting — the server **MUST NOT** read “no telemetry” as failure; it is an honest gap in the corpus, not a FAIL.
 5. **Signature verify against the per-account key** — today resolvePublicKey returns **one global** HELIX_ARTIFACT_PUBKEY (11_* §3, handlers_artifact.go:283-288, config.go:72-77). For multi-account, the verify key must become **per-account** on **both** sides: server-side upload verification (per-account key registry) **and** device-side VerifyBeforeApply (the device checks against **its account’s** public key, provisioned at enrollment **from server config** — **never from the update offer**, preserving the 11_* §3 trust boundary). 20_*/26_* own the key-registry shape.
 6. **Real artifact byte storage (dependency, 11_* honest gap 1)** — upload currently **discards the bytes** (StorageRef placeholder, handlers_artifact.go:184) and the device download URL points at an external Storage brick not in-repo (handlers_client.go:232-234). A production device client that actually downloads-and-applies **requires real per-account object storage + working (ideally signed, expiring) download URLs**. This is a server/storage prerequisite the device client depends on — flagged, not assumed solved.
-

6. Test strategy (anti-bluff, §11.4.27 / §11.4.69 / §11.4.107 / §11.4.169)

Mocks/stubs permitted **only** in unit tests (§11.4.27); every other layer drives the **real** server.

- **Unit (core Kotlin, mocks OK here only):** poll-decision logic, the `VerifyBeforeApply` ordering (`MALFORMED_DIGEST`→`HASH_MISMATCH`→`SIGNATURE_INVALID`), the wizard state machine (notify → consent → apply transitions), offline/resume state.
- **Integration (real ota-server):** drive a concrete client (extend the `ota-device-emu` harness, 11_* §4) through **login** → **register(account, project)** → **poll** → **200/204**; assert per-account key verification; **cross-account isolation test** — a device of account A polling **MUST** receive 204/deny for a deployment published under account B (captured request/response evidence proving the tenant boundary holds; a leak here is a release blocker).
- **E2E (full real journey, real content, no mock beyond unit):** provision a device to (account, project) → operator **publishes a real release under that account** → device is **notified** (push wake) / **polls** → wizard **consent** → **download** → **verify** → **apply** (emulated `update_engine`) → **telemetry** → server **derives progress**. Assert the applied version is the published one and the device on the *other* account is unaffected.
- **Push/notification latency test:** publish → wake fires → device polls within seconds (assert the hybrid §2.3 latency claim, with the poll-only floor also covered).
- **UI visual proof (§11.4.170):** wizard screens host-rendered (Compose → Paparazzi/Roborazzi) per **screen × state × {light, dark}**, golden image-diff + OCR/vision oracle — **not** value-equality tests.
- **Stress + chaos (§11.4.85):** offline mid-download → resume; server drop mid-apply; **corrupt payload** → **VerifyBeforeApply reject** (never applies); power-loss mid-apply → **A/B auto-rollback** (device still boots) — each with captured recovery evidence.
- **Captured evidence (§11.4.107 liveness / not-stale):** every PASS cites a real artefact under `docs/qa/<run-id>/` — HTTP transcripts, window-scoped vision-verified screen recording of the wizard (§11.4.154 / §11.4.158 / §11.4.160), `update_engine` apply logs, telemetry payloads; an “update applied” claim proves the **new** version is live (not the stale prior one). §11.4.185 manual QA team

confirmation is the final gate; §11.4.169 requires the full test-type set.

Sources verified 2026-07-10

- **AOSP SystemUpdaterSample (A/B update_engine reference client)** — pull model, list-of-updates UI + Apply/Stop/Reset/Suspend-Resume, UpdateEngine#applyPayload handoff with headers, STREAMING vs non-streaming, resume + onStatusUpdate / onPayloadApplicationComplete progress callbacks: https://android.googlesource.com/platform/bootable/recovery/+master/updater_sample/README.md
- **AOSP OTA / A/B (seamless) system updates** — update_engine, slot switching, auto-rollback semantics: <https://source.android.com/docs/core/ota> and <https://source.android.com/docs/core/ota/ab>
- **OTA poll-vs-push tradeoffs for device fleets** — pull = lower bandwidth but interval-bound latency + unsafe-state deferral; fleet-scale thundering-herd → floating/jittered poll cycle; FCM single shared socket = battery-efficient instant wake: Mender OTA best-practices <https://mender.io/resources/reports-and-guides/ota-updates-best-practices>, saince.io OTA update modes <https://saince.io/2020/08/24/ota-update-modes/>, Hubble “Every Way to Push OTA Updates to IoT Devices in 2026” <https://hubble.com/community/guides/every-way-to-push-ota-updates-to-iot-devices-in-2026/>
- **Multi-tenant device provisioning / enrollment** — AWS IoT *fleet provisioning by claim*: a shared, minimally-privileged bootstrap credential is exchanged at first connect for a unique per-device production credential, driven by a provisioning template + Lambda provisioning-hook validation (basis for §1.3 Option B): <https://docs.aws.amazon.com/iot/latest/developerguide/provision-wo-cert.html> and <https://aws.amazon.com/blogs/iot/how-to-automate-onboarding-of-iot-devices-to-aws-iot-core-at-scale-with-fleet-provisioning/>

Negative finding (§11.4.99): none of the sources contradicts the Helix poll-first design; AWS’s model is certificate/MQTT-based whereas Helix is bearer-token/HTTP — the *pattern* (bootstrap → rotate, tenant-id-routed provisioning) transfers, the transport does not. The A/B update_engine handoff is platform-authoritative and adopted directly.

Honest boundary (§11.4.6)

- **Design proposal only** — no code is written; the OTA multi-account work is operator-approval-gated (00_INDEX §2). Nothing here is a §11.4 PASS-bluff: current state is cited to 11_* (which cites file:line); external patterns are cited to the sources above; every unknown is flagged, never guessed.
- **20_* is the SSOT for entity shapes and is not yet authored.** This doc used only the direction 00_INDEX already fixes (account → projects → OTA, server-minted token account/project claim, scoping columns) and **invented no conflicting account/project/device shape**. Exact field names/types (token claim, store.Device columns, per-account key registry) are deferred to 20_*/21_*/26_*.
- **Real gaps that remain (not solved by this design):** (1) the **concrete device HTTP client + device config** are absent from the agent submodule — interfaces only, the Go emulator is the only concrete reference (11_* honest gap 2); (2) **real artifact byte storage** is not implemented — bytes are validated then discarded, StorageRef is a placeholder, and the device download URL points at an out-of-repo Storage brick (11_* honest gap 1) — a device that truly downloads-and-applies depends on this being built; (3) the **push/wake channel, per-account key registry, release-notes field**, and the **entire wizard UI** are net-new. These are stated as facts to be built, not as existing capability.
- **Recommendations are explicit, never silent:** Option A (operator-minted scoped token) first, Option B (bootstrap → rotate) when self-service/scale demands it (§1.3); hybrid **poll-floor + push-wake** transport, degrade to poll-only if simplicity is preferred and to SSE/WebSocket where FCM/GMS is unavailable (§2.3). The final choices are the operator's / 20_*'s to make.